GAME1402 - Assignment 1

2D Platformer Prototype

Written by Kyle MacLachlan

Core Systems

My 2D platformer is inspired by the Nintendo 64 2D Platformer, Yoshi Story. The objective of the game is to collect 30 fruits in a level, to proceed to the next level. In Yoshi Story, this objective ties into the narrative, where 6 baby yoshi's are eating fruit to be happy as they search for the Super Happy Tree, which Baby Bowser had stolen.

While my game has no narrative design, I decided I wanted to work towards the basic mechanics that Yoshi Story had designed, and, eventually expand the concept with new mechanics – Those new mechanics are only lightly treaded on in this game, since the more complex features require coding lessons we have not covered in class.

Some notable points of the game include: **The use of a controller to play the game**, **the variety of fruit present in the game**, **the random population of the different fruits**, and **the use of sprites to illustrate the game**.

The Use of a Controller to Play the Game

One of my design philosophies going into every single one of my assignments is the ability to program a controller into the game's actions. After this program, my long-term goal is to work for a larger game company – such as Nintendo, or one of their many third-party developers like Retro Studios, or Next Level Games. So having an understanding on how to program a controller to do not just the basic actions of jumping and moving, but pausing, interacting with the environment in context sensitive areas, and combat will all be important.

For this game, I have now purposefully programmed the game to run with PS5 DualSense Controllers, and the Xbox One Controller, along with the Keyboard controls to add a bit of accessibility towards playtesting this game.

The Variety of Fruit

Yoshi Story on the N64 had featured a total of 5 types of fruit: Apple, Banana, Grape, Watermelon, and Melon. There were also 6 Yoshis, all who had a "preferred fruit" programmed into the game that would increase their score if they ate their preferred fruit over ordinary fruit (ignoring the Melon for this context since that is a different kind of mechanic).

As I wanted to expand the concept of Yoshi Story, I knew I would want to expand the types of fruit in the game. While my initial workshop had worked out a total of 30 fruit, I have

instead limited my project to 17 fruit; simply due to the fact that not all the fruit I want to include in the game had sprites to work with at the time.

I have included the following fruit: (**Bold** identifies a New Fruit, *Italic* identifies a fruit that was in the original Yoshi Story, but didn't count as an edible fruit but a hazard)

Apple	Banana	**Blueberry**	**Cherry**
Dragonfruit	Grape	**Lemon**	**Lime**
Orange	**Peach**	**Pear**	*Pepper*
Pineapple	**Pomegranate**	**Raspberry**	**Strawberry**
Watermelon			

The Random Population of Fruit

With the list of available fruit now made, I wanted to be able to create a unique experience for players going through the game. When the final components were put together (AKA, the favoured fruits), players would get extra points for collecting their character's favorite fruit. To make this mechanic interesting, I wanted to have all the fruit generated randomly across the level.

SPECIAL THANKS to Indika who provided a snippet of code to apply this mechanic. Code listed on the next page.

```
using UnityEngine;

[RequireComponent(typeof(SpriteRenderer))]
[RequireComponent(typeof(Fruit))]

public class RandomSpriteOnStart : MonoBehaviour
{
   [Header("Sprites match Fruit Types by Index")]
   public Sprite[] possibleSprites;
   public string[] fruitTypes;

   void Start()
   {
      if (possibleSprites == null || possibleSprites.Length == 0 || fruitTypes.Length == 0)
      {
         Debug.LogWarning("Missing Sprites or fruit data on" + gameObject.name);
         return;
      }

      if (possibleSprites.Length != fruitTypes.Length)
      {
         Debug.LogWarning("Sprite and Fruit Type arrays must be the same length!");
         return;
      }

      int index = Random.Range(0, possibleSprites.Length);

      SpriteRenderer render = GetComponent<SpriteRenderer>();
      render.sprite = possibleSprites[index];

      Fruit fruit = GetComponent<Fruit>();
      fruit.SetFruitType(fruitTypes[index]);
   }
}
```

Sprites to Illustrate the World

Last Semester, the program taught us 2 things:

1. How to program a game in Unity
2. How to put sprites and tilesets into Unity

HOWEVER, the program had not yet had us combine the two systems to create a cohesive game that looks good and programs well. So, I decided I wanted to make this game not only program well but look good as well. I found a few tilesets on itch.io that would play a huge part in the game's main tileset, and some background and foreground elements.

I HAD planned to include parallax backgrounds to give a sense of environment. However, time constraints to this assignment have pushed that element back. There are MORE IMPORTANT GAME ELEMENTS THAT NEED TO BE INCLUDED.

Architectural Decisions

I had decided to build this game as a "light Yoshi Story" allowing me to work with the basic core mechanics of a platformer game, and be able to explore different ideas and decisions. This included Level Design, Hazards, Game Systems, and figuring out GitHub (I despise GitHub right now, knowing I need to get over this hurdle and learn to like it).

Level Design

I chose to focus my first level on a very simple Grassland stage, iconic to many of the Super Mario levels. It's intro area is very simple allowing players to learn the controls in a safe environment, and once the players leave the starting space, then hazards will start to appear. I wanted to really put my "game design" skills to the test by applying sorting layers to the level; this allowed me to place objects in front of, and behind the player, and also create the illusion of going into a large cave; because there is a vertical descent in the game, I had to identify what a Bad Pit was that lead to death, and what a Good Pit was that allowed players to continue their adventure.

After the cave, the player is introduced to split paths, branching out to dead-ends and secrets, or creating alternate routes to backtrack. This comes together to a final platforming challenge of moving platforms.

At the end of the level, the player finds a doorway, which warps the players to a point just before the start of the level; if the player had not collected 30 fruit by that point, or decided to explore the whole level, this is the loop that allows players to return to the start and continue finding more fruit.

Hazards

We don't know how to make enemies, or how to program the players to be able to attack enemies with clean and clear code. So I decided I would create "hazards", which are stand-in enemies. They cannot be defeated, and thus have a triangular sprite to illustrate – DO NOT STAND ON THIS! They move back and forth as if they were Red Koopa Troopas on the march, since we also don't know how to program enemy movements yet.

Players can get hit, and there are many Healing Hexagons the player can collect to heal themselves.

Game Systems

This one is complex, because I had to go back into the assignment after the level design was finished in order to add and build additional elements that the assignment required.

1) The Pause Menu was implemented by adding Action Assets to my controller; it pauses the player in place, stops the timer (which is explained below and built before the Pause Menu). This does not use an enum.
2) The Level Timer was built as a clock counting up rather than a clock counting down. Since Yoshi Story was a jolly romp through levels, that allowed looping in a stage, I decided to use that philosophy to allow players to explore at their leisure, and when they collect 30 fruit, they have a timed score. On second playthroughs, players can attempt to beat their score.
3) Death and Respawn – If a player takes too much damage from hazards, or fall into a pit, they lose a life. Losing a life sends the player back to the start of the level. When all lives are used, it's game over.

Key Code Snippets

A lot of my code followed along with our in-class instructions and tutorials, a lot of my code was done through research on Google, forums, and Unity's resources in order to get the code running and functional, while keeping the code uniformed to what sorts of lessons we covered in class.

GameManager Instance – Lives Sample

```
[Header("Collectables")] [SerializeField]
private int winFruitCount = 30; // sets a cap fruit amount to collect to clear the level

private int currentFruitCount = 0; // sets the current fruit count for the level to 0
private int currentCoinCount = 0; // sets the current fruit count for the level to 0

[Header("Lives")]
[SerializeField] private int maxLives = 6;
private int currentLives;

[Header("Level Timer")]
[SerializeField] private float levelTime;
[SerializeField] private bool timerRunning = true;

[Header("Pause")]
[SerializeField] private bool isPaused = false;
```

```
private void Awake()
{
   // Singleton setup
   if (Instance == null)
   {
      Instance = this;
   }
   else
      Destroy(gameObject);
}
```

```
public int GetLives()
{
   return currentLives;
}
public void LoseLife()
{
   currentLives--;
   HUDManager.Instance.UpdateLives(currentLives);

   Debug.Log($"Player Died. Lives left: {currentLives}");

   if (currentLives <= 0)
      GameOver();
}

public void GameOver()
{
   PlayerController player = FindObjectOfType<PlayerController>();
   timerRunning = false;
   Debug.Log("GAME OVER");

   // disable player movement, stop game, etc.
   player.enabled = false;
   player.HidePlayer();
   ShowTutorialMessage("----- GAME OVER -----\n \nGood try, you can always try again.");
  Time.timeScale = 0;    // pauses game.
}
```

PlayerController MonoBehavior – Knockback and Death

```
[Header("Knockback")]
[SerializeField] private float knockbackForce = 8f;
[SerializeField] private float knockbackUpForce = 4f;
```

```
public void TakeDamage(int damage)
{
   if (isDead) return;
   if (Time.time < lastDamageTime + damageCooldown) return;

   lastDamageTime = Time.time;

   currentHealth -= damage;

   HUDManager.Instance.UpdateHealth(currentHealth,  maxHealth);

   Debug.Log($"Player HP: {currentHealth}/{maxHealth}");

   if (currentHealth <= 0)
      Die();
}

public void ApplyKnockback(Vector2 sourcePosition)
{
   if (_playerRB == null) return;

   Vector2 direction = ((Vector2)transform.position - sourcePosition).normalized;
   Vector2 force = new Vector2(direction.x * knockbackForce, knockbackUpForce);

   _playerRB.linearVelocity = Vector2.zero;
   _playerRB.AddForce(force, ForceMode2D.Impulse);
}

void DeathCheck()
{
   RaycastHit2D hit = Physics2D.Raycast(
      (Vector2)transform.position + deathRayOffset,
      Vector2.down,
      deathCheckDistance,
      deathLayer
   );

   if (hit.collider != null)
   {
      Die();
   }
}

void Die()
{
   if (isDead) return;
   isDead = true;

   GameManager.Instance.LoseLife();

   currentHealth = maxHealth;     // reset player Health.

   HUDManager.Instance.UpdateHealth(currentHealth,  maxHealth);

   RespawnManager.Instance.RespawnPlayer(gameObject);

   isDead = false;
}
```

HUDMananger Instance – the complete code:

```
public static HUDManager Instance;

  [Header("Counters")]
  [SerializeField] private TextMeshProUGUI fruitText;
  [SerializeField] private TextMeshProUGUI coinText;
  [SerializeField] private TextMeshProUGUI livesText;
  [SerializeField] private TextMeshProUGUI healthText;

  [Header("Tutorial UI")]
  [SerializeField] private TextMeshProUGUI tutorialText;

  [Header("Timer")]
  [SerializeField] private TextMeshProUGUI timerText;

  private void Awake()
  {
    if (Instance == null)
      Instance = this;
    else
      Destroy(gameObject);
  }

  // Update Methods
  public void UpdateFruit(int current, int win)
  {
    fruitText.text = current +  " / " + win;
  }

  public void UpdateCoin(int value)
  {
    coinText.text = value.ToString();
  }

  public void UpdateLives(int value)
  {
    livesText.text = value.ToString();
  }

  public void UpdateHealth(int current, int max)
  {
    healthText.text = current + " / " + max;
  }

  // Tutorial Methods
  public void ShowTutorialMessage(string message)
  {
    tutorialText.text = message;
    tutorialText.gameObject.SetActive(true);
  }

  public void HideTutorialMessage()
  {
    tutorialText.gameObject.SetActive(false);
  }

  // Timer Methods
  public void UpdateTimer(float time)
  {
    int minutes = Mathf.FloorToInt(time / 60f);
    int seconds = Mathf.FloorToInt(time % 60f);

    timerText.text = $"Time: {minutes:00}:{seconds:00}";
  }
}
```

Debugging Challenges

While working with the randomized fruit, I realized I needed a way for the game to identify what fruit I was collecting each time; so I used Debug.Log in order to help me figure out what sort of code and syntax I needed to use.

I used Debug to try to figure out how to apply and create Run as a control scheme. This however did not help with programming running into the game; I ended up dropping this mechanic.

Because all my code is set up modularly – each key piece of code is in it's own coded document, I used Debug to ensure communication between the codes are being executed. Collecting a fruit sends code from my collectable's individual script, into the collector, into the ICollectable, and into the HUDManager.

AI Tool Usage (Minor use):

Disclaimer: I did use AI to help better understand the syntax and commands I used for some of the more complex game elements that were not taught in class, such as my Respawner – I used AI to help explain why I shouldn't Destroy(gameObject) and how to get the player to respawn back at start; I also found out that even on GameOver, I should not Destroy(GameObject), so I then went to google to find code to help hide the player at game over instead.

The other element I used AI to help with was entering doors. I had asked about how I can get my player to interact with a door to warp to another door on the other side of the room, and it helped guide some basic code that worked with my Action Assets. There is more than likely a cleaner solution,